Table of Contents

Week 4 — Summary & Examples Reference

Alternate Real-World Examples for Every Part of the Lesson

Use alongside: Week4_Encapsulation.md | **Class:** Wednesday, 24 June 2026 · 6:30 PM

> *This file mirrors the main lesson guide part-by-part, but every example here is **different** from the main file. Use these if a student needs a second angle, or to add variety mid-discussion without repeating yourself.*

One-Line Definitions (Read These Aloud)

> ***Encapsulation** means bundling an object's data with its actions, then hiding that data so it can only be reached through approved, controlled actions.*

> ***Private** means only the object itself can directly touch its own data.*
> ***Public** means any outside code can directly use it.*

> ***A getter** safely reports information without changing anything.*
> ***A setter** safely requests a change — but checks the request first.*

> ***Validation** is the check a setter performs before accepting a new value.*

Part 1 Alternate Examples — What Is Encapsulation?

> *Main file used: the ATM machine + the childproof medicine cap.*
> ***This file uses:** a car's dashboard vs engine, and a restaurant kitchen.*

Alternate Analogy — Driving a Car

```
DRIVING A CAR
──────────────────────────────────────────────────────────────
WHAT YOU INTERACT WITH (the public interface):
```

```
  → Pedals (accelerator, brake)
    → Dashboard (speed, fuel level, warning lights)

WHAT YOU NEVER NEED TO TOUCH (the hidden internals):
  → The engine's internal combustion process
  → The transmission's gear-shifting mechanism
  → The fuel injection system
  → The wiring behind the dashboard

WHY DOES THIS DESIGN EXIST?
  → Safety — you cannot accidentally damage the engine while driving
  → Simplicity — you only need to learn a few controls, not the
    entire mechanical system
  → Reliability — the car's internal systems work the same way
    every time, regardless of who is driving

This is encapsulation: a small, simple public interface —
with a complex, protected system hidden behind it.
──────────────────────────────────────────────────────────────
```

The Problem Story — A Restaurant Kitchen With No Rules

> *"Imagine a restaurant where any customer could walk straight into the kitchen, stir whatever pot they liked, change the spice levels, or move the chef's tools around — all while the chef is in the middle of cooking.*
>
> *Would you trust the food coming out of that kitchen? Would the chef ever be able to guarantee consistent quality?"*

```
THE PROBLEM:
──────────────────────────────────────────────────────────────
The kitchen's internal process can be disturbed by anyone,
at any time, with no rules and no oversight.
The chef (the object) cannot guarantee the quality or safety
of what they produce.
──────────────────────────────────────────────────────────────
```

> *"This is resolved by giving customers a **menu** and a **waiter** — a public interface. Customers order through the menu; the kitchen stays protected and consistent behind the scenes. This is encapsulation in a restaurant."*

Part 2 Alternate Examples — Public vs Private: Two Levels of Access

> *Main file used: a house (living room vs bedroom) + a comparison table (house/phone/school/diary).*
> ***This file uses:*** *a vending machine and a mailbox.*

Alternate Analogy — A Vending Machine

```
VENDING MACHINE
──────────────────────────────────────────────────────────────

PUBLIC (anyone can use these directly):
  → The coin slot
  → The selection buttons
  → The item collection window

PRIVATE (hidden inside the machine):
  → The mechanism that counts and verifies coins
  → The internal stock count for each snack
  → The dispensing mechanism that releases the item

A customer never opens the machine to grab a snack directly,
and never reaches in to fix the coin counter. They use the
public buttons and slot — the machine handles the rest.
──────────────────────────────────────────────────────────────
```

Alternate Analogy — A Mailbox (Showing One-Way Access)

> *"A mailbox is an interesting case — access isn't always symmetric. Anyone walking by can PUT mail IN through the slot (this is like public write access, in one direction). But only the postal worker with a key can TAKE mail OUT (this is restricted, like private read access in the other direction)."*

```
MAILBOX
──────────────────────────────────────────────────────────────

Putting mail IN:    PUBLIC — anyone can use the slot
Taking mail OUT:    PRIVATE — only the key-holder can open it

This shows: public and private don't have to be "all or nothing."
An object can choose to allow one kind of access while
restricting another — exactly like designing only a getter,
or only a setter, for a particular piece of information.
──────────────────────────────────────────────────────────────
```

Part 3 Alternate Examples — Read Access, Write Access & Validation

> *Main file used: the childproof cap (revisited), the ATM withdrawal limit, and the school GPA range.*
> ***This file uses:*** *a thermostat and a cinema age check.*

Alternate Analogy — A Thermostat With Limits

```
HOME THERMOSTAT
──────────────────────────────────────────────────────────────

GETTER (read access):
  "What is the current temperature?" → reports 24°C
  Just reading the display changes nothing.

SETTER (write access, WITH validation):
  "Set the temperature to 24°C" → accepted, system adjusts
  "Set the temperature to 500°C" → REJECTED — physically
                                   impossible / dangerous
  "Set the temperature to −50°C" → REJECTED — outside the
                                   safe operating range

The thermostat protects itself (and your home) by refusing
requests that don't make sense — exactly like a setter
validating its input before accepting a change.
──────────────────────────────────────────────────────────────
```

Alternate Analogy — Cinema Age Verification

```
BUYING A TICKET FOR AN 18+ MOVIE
──────────────────────────────────────────────────────────────

GETTER-LIKE ACTION:
  Checking the movie's age rating on the poster — read-only,
  nothing changes.

SETTER-LIKE ACTION (WITH validation):
  You tell the system your age to buy a ticket.
  Age = 25  → accepted, ticket sold
  Age = 12  → REJECTED — does not meet the 18+ requirement

The cinema's ticketing system doesn't just trust any number
typed in — it checks the request against a rule before
accepting it. This is exactly what setter validation does.
──────────────────────────────────────────────────────────────
```

Part 4 Alternate Example — Before and After Encapsulation

> *Main file used: Sokha's Bank Account (deposit/withdraw/balance protection).*
> ***This file uses:*** *a Library Book Checkout System — recall the* `Book` *class example from Week 2.*

A Library's Book Checkout System — Before Encapsulation (Open Access)

```
SCENARIO                                  WHAT COULD GO WRONG
──────────────────────────────────────────────────────────────────────────
Any code can directly set a book's        Two different readers both think
"isAvailable" to true, even while         they have legitimately borrowed
someone else is still reading it          the same physical book

Any code can directly set                 A book that was never checked
"borrowerName" to anything                out shows a random person's
                                           name as the current borrower

There is no record of WHEN a book         No way to know if a book is
was checked out or returned               overdue, or for how long
──────────────────────────────────────────────────────────────────────────
```

A Library's Book Checkout System — After Encapsulation (Protected Access)

```
SCENARIO                                  WHAT HAPPENS NOW
──────────────────────────────────────────────────────────────────────────
Outside code tries to directly            Rejected immediately — not
set "isAvailable" to true                 possible. Must use checkOut()
                                           or returnBook() instead

Someone tries to check out a              The checkOut() action checks
book that is already checked out          "isAvailable?" first and
                                           refuses if it's already taken

Someone tries to return a book            The returnBook() action checks
that was never checked out                whether it was actually checked
                                           out, and handles it sensibly
──────────────────────────────────────────────────────────────────────────
The book object now protects its own record.
No code anywhere else in the library system can create a
double-booking or a phantom borrower.
```

Discussion Prompt

> *"If `isAvailable` were public, what is the very first thing that could go wrong on a busy day with many readers and many books? How does encapsulation prevent that specific problem?"*

Part 5 Alternate Example — Java as Illustration

> *Main file showed a light `BankAccount` class.*
> ***This file shows the same concept using the Library Book example.***

```java
// Encapsulated Book – the Library checkout concept made concrete

public class Book {

    // PRIVATE attributes – hidden, protected
    private String title;
    private boolean isAvailable;
    private String borrowerName;

    public Book(String title) {
        this.title = title;
        this.isAvailable = true;       // every new book starts available
    }

    // PUBLIC read access (getter)
    public boolean isAvailable() { return isAvailable; }
    public String  getTitle()    { return title; }

    // PUBLIC write access, WITH validation
    public void checkOut(String borrower) {
        if (!isAvailable) {
            System.out.println(title + " is already checked out.");
            return;                    // reject the bad request
        }
        isAvailable   = false;
        borrowerName  = borrower;
        System.out.println(title + " checked out to " + borrower);
    }

    public void returnBook() {
        if (isAvailable) {
            System.out.println(title + " was not checked out.");
```

```
        isAvailable  = true;        borrowerName = null;
        System.out.println(title + " has been returned.");    }    }
```

```
Book novel = new Book("The Art of War");

// novel.isAvailable = true;   ← this line would not even compile!
//                                'isAvailable' is private

novel.checkOut("Sokha");
novel.checkOut("Dara");        // rejected — already checked out
novel.returnBook();
novel.checkOut("Dara");        // now succeeds
```

Concept Translation Table

```
Java code                  OOP concept meaning
─────────────────────      ──────────────────────────────────────
private isAvailable        Hidden — only this class can touch it
isAvailable()               Public read access — safely ask for it
checkOut(borrower)          Public write access — request a change
if (!isAvailable) return;   Validation — the rule checked first
novel.isAvailable = true    Not even possible — proves the data
  (commented out)            is truly protected
─────────────────────      ──────────────────────────────────────
```

Public or Private? — Quick Test Practice Set

Use these fresh items (different from the main file's challenges) for a rapid-fire class activity.

Item	Answer	Why
A car's current speed reading on the dashboard	Public (read-only)	Drivers need to see this freely — but only the car's own systems should change it
A car's engine ignition timing	Private	Only the engine's own internal systems should touch this — never a driver, directly

A vending machine's internal stock count	Private	Only the machine's own restocking process should update this
A library's catalogue of book titles	Public (read-only)	Everyone should be able to browse and search freely
A library book's borrower history	Private	Only the library's own checkout system should record and update this

---

Validation Scenarios — Use in Class Discussion

Scenario 1 — The Vending Machine Stock

```
Object: Vending Machine
Attribute: stockCount for "Chips" = 3

Event: A buyer tries to buy "Chips" when stockCount = 0

Question: What should the validation check, and what should happen?
Answer:   The dispense() action checks "is stockCount > 0?" first.
          If not, it refuses and shows "Out of Stock" — it does NOT
          pretend to dispense a snack that doesn't exist.
```

Scenario 2 — The Thermostat Range

```
Object: Thermostat
Attribute: targetTemperature

Event: Someone tries to set the temperature to −50°C

Question: Should this be accepted? What rule applies?
Answer:   Rejected. A reasonable validation rule might be:
          "targetTemperature must be between 16°C and 30°C."
          Anything outside that range is refused.
```

Scenario 3 — The Library Double-Checkout

```
Object: Book ("The Art of War")
State:  isAvailable = false (already checked out to Dara)
```

```
Question: What does checkOut() do, and why does this matter?
Answer:   checkOut() checks "isAvailable?" first. Since it's
          false, the request is refused. Without this check,
          BOTH Sokha and Dara would believe they have the book
          – a real-world conflict the object itself prevents.
```

Student Discussion Questions

1. "Give one example from your own life of something that is 'public' in the sense we discussed today — anyone can use it directly, no checks needed."
2. "Give one example of something 'private' that you interact with only through an approved action (like a bank vault through an ATM, or stock in a vending machine through its buttons)."
3. "Why is a setter with NO validation basically the same as making the attribute public? What is actually being protected?"
4. "Think of the mailbox example — can you think of another real-world object where 'putting something in' is easy (public) but 'taking something out' is restricted (private)?"
5. "In the library example, what would happen over a whole semester if `isAvailable` had no protection at all? Would the problem get better or worse over time?"

Common Student Confusions — Quick Fixes

If a student says...	Respond with...
"Private means nobody can ever use it"	"Like a vending machine's stock — YOU can't reach in and grab a snack, but the machine's own dispensing action can. Private means no DIRECT outside access — not 'frozen forever.'"
"A getter and a setter are the same thing"	"A getter only *reports* — like checking a thermostat's current temperature. A setter *requests a change* — like trying to set a new temperature. Reading and changing are different actions."
"Validation is just extra, unnecessary work"	"Without it, a thermostat would accept '500 degrees.' Validation is what makes a setter actually useful — not just decoration."

"If I can see something, I should be able to change it directly"	"You can SEE the vending machine's snacks through the glass — that's read access. You still can't reach in and grab one — you must use the public buttons. Seeing isn't the same as having direct write access."

Lesson Arc Summary (Mapped to Both Examples)

```
HOW THE TWO EXAMPLE SETS LINE UP, PART BY PART
──────────────────────────────────────────────────────────────────
PART                  MAIN FILE EXAMPLE        THIS FILE'S EXAMPLE
──────────────────────────────────────────────────────────────────
Part 1 – Concept       ATM machine +             Car dashboard/engine +
                        Medicine cap              Restaurant kitchen

Part 2 – Public/       House (rooms) +           Vending machine +
Private                comparison table          Mailbox (one-way access)

Part 3 – Getters/      Childproof cap +          Thermostat +
Setters/Validation     ATM limit + School GPA    Cinema age check

Part 4 – Before/       Sokha's Bank Account      Library Book
After Walkthrough      (deposit/withdraw)        Checkout System

Part 5 – Java          BankAccount class         Book class
Illustration           (balance, deposit)        (isAvailable, checkOut)
──────────────────────────────────────────────────────────────────
Same concepts. Different stories. Use whichever lands best
with your students – or use both, side by side, for reinforcement.
```

Homework Reminder

> ***No Java code required this week (optional bonus only).***

Continue the `Smartphone` design from Weeks 2–3:

1. Make all attributes private
2. Design a getter for battery level
3. Design a charging action with validation: battery cannot exceed 100% or drop below 0%
4. Write a short before/after comparison explaining what the validation rule prevents

---